

Mumbai MuleSoft Meetup (In Collaboration with Guwahati Meetup Group)

MuleSoft Training for Salesforce Developers and
Beginners - Module 11



Date: 20th Dec 2025
Time: 11 AM to 1 PM



Safe Harbour Statement

- Both the speaker and the host are organizing this meet-up in individual capacity only. We are not representing our companies here.
- This presentation is strictly for learning purposes only. Organizer/Presenter do not hold any responsibility that same solution will work for your business requirements.
- This presentation is not meant for any promotional activities.





Housekeeping



A recording of this meetup will be uploaded to events page within 24 hours.



Questions can be submitted/asked at any time in the Chat/Questions & Answers Tab.

Make it more **Interactive!!!**



Give us feedback! Rate this meetup session by filling feedback form at the end of the day.

We Love Feedbacks!!! Its Bread & Butter for Meetup.



Moderators/Speakers



Jitendra Bafna

Senior Solution Architect
EPAM Systems

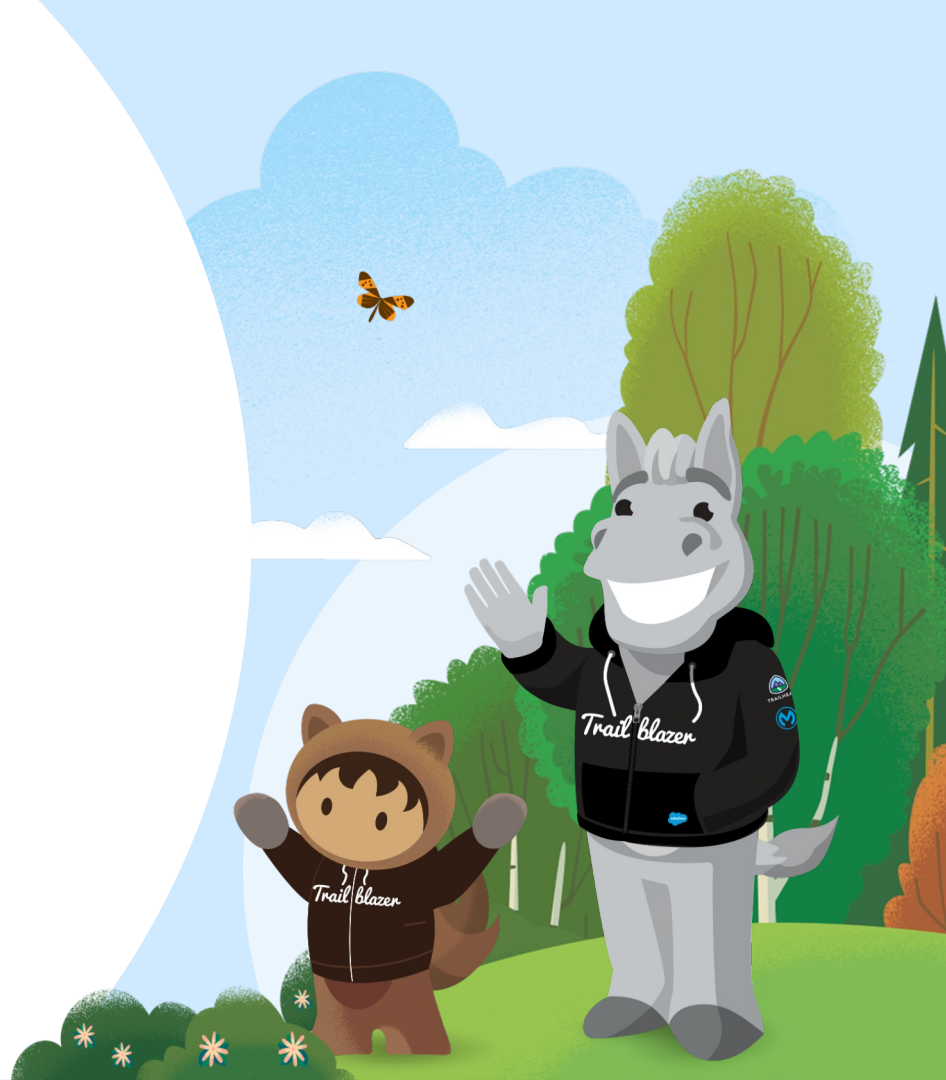


Abhishek Bathwal
Technical Architect
NeuraFlash



Agenda

Introduction To MUnit
Different Approaches of Writing MUnit
Limitations of Test Recorder
Munit Tools and Components
Executing MUnit Suits
Munit Coverage
Idempotent Message Validator
First Successful
Round Robin
Q&A



What is Munit?

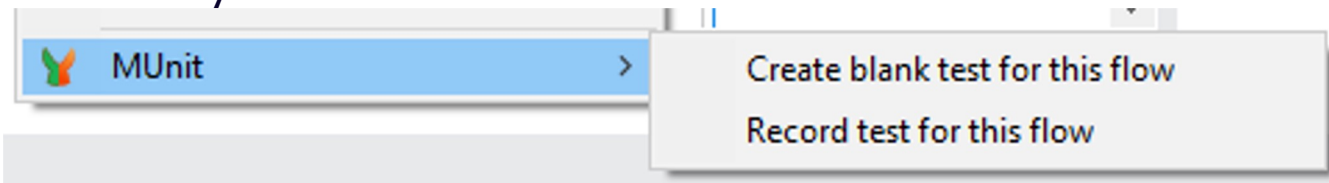


MUnit is Anypoint Platform's dedicated unit testing tool specifically designed for Mule applications. It runs inside a Mule runtime, manually in Anypoint Studio, and automatically as part of the Maven-based CI/CD build process. MUnit is designed to create structural test cases as Mule flows. That means you can use components on top of MUnit and MUnit tool modules.



Different Approaches of Writing Munit

- Munit Test Recorder
- Manually



The following table describes the product versions required to use the test recorder.

Product	Version
Anypoint Studio	7.5.0 and later
MUnit	2.2.5 and later
MUnit Anypoint Studio Plugin	2.5.0 and later
Mule runtime engine	4.3.0 and later



Limitations with Test Recorder

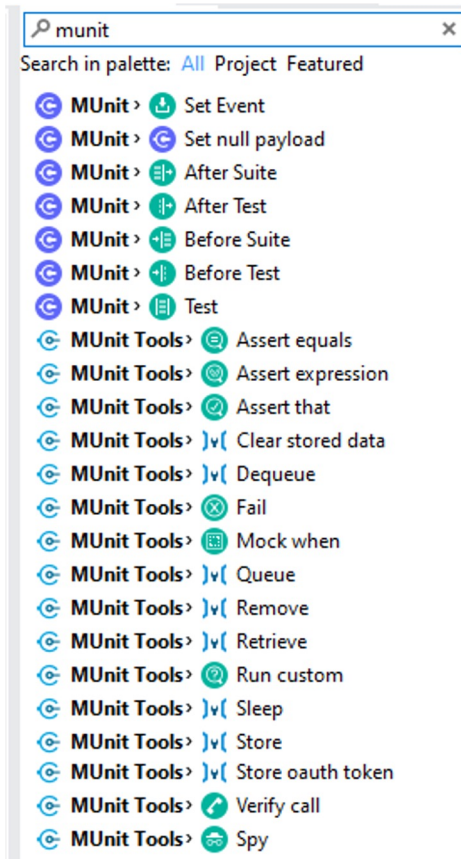
The test recorder is limited in the following ways when you create MUnit tests:

- You cannot create tests for flows with Mule errors raised inside the flow or already existing in the incoming event, not even if they are handled by an on error continue error handler.
- A recorded flow execution ends successfully, but the result does not reach its destination because the application is killed.
- Your validations fail every time your test runs if you configure Spy or Assert processors to assert values for random data, time-dependent information (such as timestamps), or values resulting from parallel processes, because those values change in every execution.
- Mocking values resulting from parallel processes causes a mixture of real and mocked data that compromises the execution of the processors that follow in your test.
- Although the recorder supports data iteration in the flow, such as recursivity or loops, it does not support cases in which the structure of the data being tested changes inside the iteration.
- The recorder does not support mocking a message before or inside a Foreach processor.



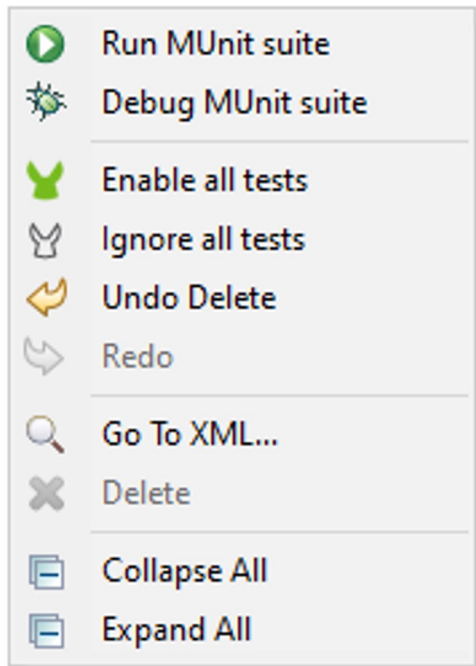
Munit Tools and Components

- Test
- Mock When
- Set Event
- Asset That
- Assert Equal
- Asset Expression
- Verify Call
- Spy



Executing MUnit Suite

- From Studio
- From cmd (mvn clean test)



```
6\munitproject-1>mvn clean test
```



Munit Coverage

The MUnit Coverage feature provides a measurement of how much of a Mule application has been executed by a set of MUnit tests by counting the amount of event processors executed. This allows you to obtain a quality standard for your application, and provides metrics for:

- Application overall coverage: An average of the items below.
- Resource coverage: Refers to each Mule configuration file under `src/main/mule`.
- Flow coverage: Refers to any of the following components: Flows, Sub-flows, and Batch jobs.



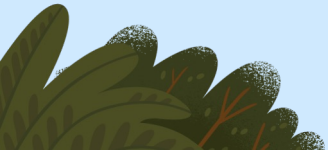
Ignoring Files or Flows

- `<ignoreFlows>`
 `<ignoreFlow>FlowName</ignoreFlow>`
 `</ignoreFlows>`
- `<ignoreFiles>`
 `<ignoreFile>FileNameet.xml</ignoreFile>`
 `</ignoreFiles>`

```
<configuration>
  <coverage>
    <ignoreFiles>
      <ignoreFile></ignoreFile>
    </ignoreFiles>
    <ignoreFlows>
      <ignoreFlow></ignoreFlow>
    </ignoreFlows>
    <runCoverage>true</runCoverage>
    <failBuild>true</failBuild>
    <requiredApplicationCoverage>75</requiredApplicationCoverage>
    <requiredResourceCoverage>50</requiredResourceCoverage>
    <requiredFlowCoverage>50</requiredFlowCoverage>
    <formats>
      <format>console</format>
      <format>html</format>
      <format>json</format>
      <format>sonar</format>
    </formats>
  </coverage>
</configuration>
```



Demo



Idempotent Message Validator

A MuleSoft component that prevents duplicate processing

- It checks if a request was already processed
- If duplicate → stops the flow
- If new → allows the flow

Why do we need it?

- Avoid duplicate records
- Prevent double payments
- Handle retries from clients or systems
- Improve data consistency



Idempotent Message Validator

How does it work? (Simple Flow)

1. Mule receives a request
2. Idempotent Validator checks a unique key
 - Example: Order ID, Transaction ID
3. If key is new
 - Request is processed
 - Key is stored
4. If key already exists
 - Request is rejected as duplicate



First Successful Router

- It routes the same message sequentially to multiple routes (processors or flows).
- Mule tries route 1 first.
- If that route fails (throws an exception), Mule moves to route 2, then route 3, and so on.
- As soon as one route succeeds, processing stops and the result from that route is returned.
- If all routes fail, the router throws an exception.

Typical use cases

- Failover logic (primary system → backup system)
- Service redundancy
- Trying multiple endpoints until one responds successfully



Round Robin Router

In MuleSoft, the Round Robin Router is used to distribute messages evenly across multiple routes in a rotating order.

What the Round Robin Router does

- Sends each incoming message to one route at a time
- Routes are chosen in a circular sequence
- First message → route 1
- Second message → route 2
- Third message → route 3
- Then cycles back to route 1
- This helps balance load across multiple systems or services.



Round Robin Router

Common use cases

- Load balancing across identical endpoints
- Distributing processing across multiple flows
- Improving throughput and availability



References

- <https://docs.mulesoft.com/mule-runtime/3.6/munit>
- <https://docs.mulesoft.com/munit/2.3/test-recorder>
- <https://blogs.mulesoft.com/dev-guides/how-to-tutorials/munit-testing-strategies/>
- <https://dzone.com/articles/mock-apikitrouter-and-write-munit-test-cases-for-d>

